FIXPOINT SWARM

AinSoft-R

Nullcenter-Hypertorus Trading Mirror

Purpose

This document evolves FIXPOINT SWARM-R v2.4.0 into an agent-executable implementation constitution. The architecture remains the same, but the document is now explicitly shaped as a direct build specification for an autonomous coding agent. The document is therefore written not only to describe the system, but to remove ambiguity about repository structure, implementation phases, mandatory files, crate contracts, testing obligations, CLI behavior, artifact schemas, and completion conditions.

System class	Deterministic Rust-native trading mirror with nullcenter-governed execution
Primary objective	Converge toward stable, replayable, net-positive execution motifs
Primary upgrade over v2.4.0	Direct coding-agent implementation constitution
Target implementation	Autonomous GitHub agent, pure Rust workspace, single-operator viable
Prepared for	Sebastian Klemm
Generated on	10 March 2026

Canonical intent: the document is now directly handoff-ready to a coding agent that must build the software autonomously without relying on hidden design interpretation.

Contents

1 Executive synthesis

Version 2.4.0 made the architecture resource-minimal and explicitly solo-operable. Version 2.5.0 keeps the exact same machine identity and makes the specification executable as a repository-building instruction set.

The system still consists of:

1. tripolar regimes,
2. tri-carrier time,
3. nullcenter-factored jumps,
4. bounded trumpet multiplexing,
5. dual-consensus mirror gating,
6. CSP execution,
7. Mirror-Hedge,
8. Shadow-Chain and Commitment Chain,
9. calibration shell,
10. promotion path,
11. integrity and recovery constitution,
12. resource-governed degradation.

What changes in this version is that the architecture is now additionally bound by:

1. a direct coding-agent mission,
2. a repository realization contract,
3. mandatory crate and file structure,
4. implementation order and phase closure,
5. mandatory test suites,
6. mandatory CLI commands,
7. mandatory documentation outputs,
8. a definition of done stated in buildable repository terms.

2 Canonical continuity rule

Continuity rule

Version 2.5.0 is the same specification at a higher maturity level. It is valid only insofar as it reduces ambiguity for autonomous implementation and increases repository realizability without altering the architectural identity of the system.

3 Why this next step is necessary

The strongest remaining weakness after v2.4.0 was not architecture, legality, or resource logic. It was handoff precision. The system was already strong enough to build, but still left too much implicit for a coding agent in:

1. exact repository generation order,
2. mandatory module boundaries,
3. minimum file set,
4. test and CI obligations,
5. implementation-stage closure,
6. required CLI behavior,
7. expected repository artifacts after completion.

Version 2.5.0 resolves that by making the document itself a direct implementation constitution.

4 Canonical design objective

Definition 4.1 (Fixpoint trading attractor)**.** A fixpoint trading attractor is a bounded family of trading motifs, hedge responses, temporal schedules, and promoted configurations such that repeated iteration of observation, multiplexing, gating, execution, hedge, calibration, replay, controlled recovery, and resource-governed operation drives the system toward the same small set of profitable and stable behaviors.

Definition 4.2 (Leverage density)**.** Leverage density is the ratio between realized admissible financial result and the total cost of compute, storage, complexity, operator attention, and capital exposure required to produce it.

The target remains:

1. stable edge recurrence,
2. bounded drawdown,
3. deterministic replay,
4. temporal coherence,
5. safe actuation,
6. exact operational legality,
7. bounded failure and reversible recovery,
8. maximal leverage density under minimal infrastructure,
9. direct autonomous implementability.

Primary doctrine

A feature that improves local sophistication while increasing interpretation burden for the coding agent is a regression unless it is strictly required for correctness or integrity.

5 Direct coding-agent mission

5.1 Primary mission

A coding agent receiving this document must generate a complete pure-Rust repository that compiles, tests, replays, simulates, validates promotions, validates rollback targets, and exposes the required CLI surface without requiring further architectural documents.

5.2 Primary priorities

When architectural freedom remains, the coding agent **MUST** choose in this order:

1. determinism,
2. safety,
3. replayability,
4. correctness of state transitions,
5. bounded infrastructure,
6. simplicity,
7. performance,
8. extensibility.

5.3 Forbidden agent behaviors

The coding agent **MUST NOT**:

1. add Python as a runtime dependency,
2. add a GUI,
3. require distributed infrastructure,
4. introduce hidden background services,
5. replace explicit state machines with opaque abstraction layers,
6. silently weaken event completeness or replay surfaces,
7. add uncontrolled machine-learning components,
8. introduce external dependencies that are not justified by correctness or integrity.

6 Retained binding core

The following remain binding:

1. regime states `Alpha/Beta/Gamma`,
2. resonance metrics $(\kappa, H, sync, M)$,
3. Spiral Index with leakage penalty,
4. hard candidate filter chain,
5. CSP state machine,

6. Mirror-Hedge FSM,
7. Shadow-Chain,
8. Commitment Chain,
9. tri-carrier temporal logic,
10. nullcenter factoring,
11. trumpet multiplexing,
12. dual-consensus mirror gate,
13. calibration shell,
14. promotion FSM,
15. integrity posture state and recovery constitution,
16. resource posture state and degradation constitution.

7 System architecture

7.1 Top-level planes

Plane	Function
Observation Plane	Market ingest, book normalization, benchmark extraction, route discovery
Temporal Plane	Tri-carrier scheduler, phase windows, horizon keys, temporal commitments
Multiplex Plane	Trumpet remap, layered candidate expansion, route-family compression
Consensus Plane	Regime gate, PoR, mirror consensus, CSP quorum, actuation admissibility
Execution Plane	Lockstep execution, hedge activation, venue adapters, receipts, abort control
Calibration Plane	Configuration contraction, phase-shifted retuning, acceptance evidence
Integrity Plane	Invariants, schema versioning, failure classification, rollback admissibility
Audit Plane	Shadow-Chain, commitment chain, snapshots, replay artifacts, metric lineage
Resource Plane	Resource budgets, degradation modes, storage rotation, solo-operator observability

7.2 Canonical macro-cycle

Listing 1: Canonical macro-cycle

```
observe -> normalize -> extract metrics
-> update (t1, t2, phi, temporal key)
-> update resource posture
-> if resource posture unhealthy:
       degrade noncritical paths first
```

```
-> select active windows and trumpet layers
-> expand route families
-> compress by invariant filter
-> evaluate gate stack
-> if all required gates pass:
       open CSP intents
       if quorum passes:
           execute lockstep
       else abort with witness
   else route to EQ / hedge / passive mode
-> update hedge
-> append shadow and commitment evidence
-> evaluate calibration shell
-> if calibration accepted:
       emit promotion candidate
-> validate invariants
-> if integrity degraded:
       enter safe-hold or rollback path
-> rotate snapshots and compact low-value artifacts if policy allows
-> publish bounded reports
```

8 Global invariants

8.1 Canonical invariant register

The following invariants remain binding:

ID	Invariant	Class
INV-01	No execution without nullcenter certificate.	blocking
INV-02	No state transition without typed event.	blocking
INV-03	No consequential event without temporal key.	blocking
INV-04	No quorum without edge, TTL, depth, slip, mirror, and temporal validity.	blocking
INV-05	No live promotion without paper validation and promotion gate.	blocking
INV-06	No replay-success claim if divergence exists and is undisclosed.	blocking
INV-07	No hidden mutable state may affect decisions.	blocking
INV-08	Shadow-Chain and Commitment Chain digests must remain hash-consistent.	blocking
INV-09	Schema migration must not destroy replay lineage.	blocking
INV-10	Hedge leverage may not exceed configured caps.	blocking
INV-11	Candidate ranking may not rescue a hard-filter failure.	blocking
INV-12	Safe-hold must freeze actuation while preserving observation and audit.	safety
INV-13	Resource degradation must never silently weaken blocking gates.	blocking
INV-14	Resource exhaustion may degrade breadth, never legality.	blocking
INV-15	The repository must remain buildable without enterprise infrastructure assumptions.	blocking

Invariant law

Any blocking invariant violation invalidates normal execution and requires safe-hold, rollback, or kill according to failure and recovery class.

9 Formal state model

9.1 State bands

1. **Operational state** x: live routing, books, inventory, metrics
2. **Calibration state** c: strategy and threshold configuration
3. **Evidence state** e: append-only event word plus hash head
4. **Commit state** ℓ: irreversible commitments, fills, receipts, accepted promotions
5. **Integrity state** i: invariant posture, schema posture, replay posture, recovery posture
6. **Resource state** r: CPU posture, memory posture, I/O posture, network posture, persistence posture

9.2 Discrete state machines

Regime state:

1. `Alpha`
2. `Beta`
3. `Gamma`

CSP state:

1. `Idle`
2. `Discovered`
3. `IntentOpen`
4. `IntentQuorum`
5. `ExecLockstep`
6. `Confirm`
7. `Settled`
8. `Abort`
9. `Reset`

Mirror hedge state:

1. `Safe`
2. `Armed`
3. `Triggered`
4. `Disarmed`

PoR FSM state:

1. `Search`
2. `Lock`
3. `Verify`
4. `Commit`

Promotion FSM state:

1. `Candidate`
2. `PaperValidated`
3. `PromotionPending`
4. `LiveEligible`
5. `LiveActive`
6. `Demoted`
7. `Revoked`

Integrity posture state:

1. `Healthy`
2. `Degraded`
3. `SafeHold`
4. `RollbackPending`
5. `Killed`

Resource posture state:

1. `Nominal`
2. `Constrained`
3. `Scarce`
4. `Emergency`

10 State transition tables

10.1 Regime transition table

From	To	Guard	Required event
Alpha	Beta	SI weakens or mirror degrades but gamma trigger not met	RegimeChanged
Alpha	Gamma	SI below trigger or drawdown above max or entropy collapse	RegimeChanged
Beta	Alpha	SI recovery, mirror consistent, risk safe	RegimeChanged
Beta	Gamma	trigger condition fires	RegimeChanged

Gamma	Beta	partial recovery without full alpha conditions	RegimeChanged
Gamma	Alpha	full recovery and hedge inactive or safely closed	RegimeChanged

10.2 CSP transition table

From	To	Guard	Required event
Idle	Discovered	candidate survives hard filters	CandidateDiscovered
Discovered	IntentOpen	all required gates pass	IntentOpened
IntentOpen	IntentQuorum	every leg satisfies quorum rules	QuorumPassed
IntentOpen	Abort	TTL, depth, slip, edge, mirror, or temporal fail	QuorumFailed
IntentQuorum	ExecLockstep	lockstep admissibility true	ExecutionStarted
ExecLockstep	Confirm	execution receipts available	ExecutionStarted
Confirm	Settled	all required confirmations valid	ExecutionSettled
Any active	Abort	explicit abort class emitted	ExecutionAborted
Abort	Reset	abort witness appended	ResetEvent
Reset	Idle	state cleanup complete	ResetEvent

10.3 Hedge transition table

From	To	Guard	Required event
Safe	Armed	arm conditions satisfied	HedgeArmed
Armed	Triggered	trigger conditions satisfied	HedgeTriggered
Triggered	Disarmed	unwind complete or forced stop	HedgeRecovered or HedgeClosed
Disarmed	Safe	no residual exposure and recovery stable	HedgeRecovered

10.4 Promotion transition table

From	To	Guard	Required artifact/event
Candidate	PaperValidated	paper suite passes	CalibrationAccepted + paper report
PaperValidated	PromotionPending	promotion proposal emitted	PromotionProposal
PromotionPending	LiveEligible	required checks pass	PromotionGatePass

LiveEligible	LiveActive	live activation approved	PromotionActivated
LiveActive	Demoted	live regression or governance decision	PromotionDemoted
Any non-revoked	Revoked	fatal issue or explicit revocation	PromotionRevoked

10.5 Integrity posture transition table

From	To	Guard	Required event
Healthy	Degraded	non-blocking failure or SLO breach	IntegrityDegraded
Degraded	SafeHold	blocking invariant breach without irreversible corruption	SafeHoldEntered
Degraded	RollbackPending	promotion regression or config fault with valid rollback target	RollbackRequested
SafeHold	Healthy	issue corrected and replay/integrity checks pass	SafeHoldReleased
RollbackPending	Healthy	rollback completed and validated	RollbackCompleted
Any	Killed	unrecoverable integrity violation or operator kill	KillActivated

10.6 Resource posture transition table

From	To	Guard	Required event
Nominal	Constrained	one or more resource budgets approach soft limit	ResourceDegraded
Constrained	Scarce	multiple budgets exceed soft limit or one exceeds hard limit	ResourceDegraded
Scarce	Emergency	legality-preserving operation at risk	ResourceEmergency
Emergency	Scarce	emergency cleared, hard legality preserved	ResourceRecovered
Scarce	Constrained	usage returns below hard limits	ResourceRecovered
Constrained	Nominal	sustained healthy budget margin	ResourceRecovered

Transition law

Every transition above is binding. No implementation may add hidden success paths, hidden recovery paths, silent resource modes, or silent terminal states.

11 Core mathematics

11.1 Resonance metrics

Let route-layer observations produce normalized route signals r_i.

$$\kappa = \text{coherence}(r_1, \ldots, r_n), \qquad H = \text{entropy}(r_1, \ldots, r_n), \qquad sync = \frac{1}{n}\sum_i r_i$$

$$M = \tanh(2 \cdot sync - 1)$$

11.2 Extended Spiral Index

$$SI = \Psi \cdot \kappa \cdot \max(0, M) - (H + fees + slip + gas + latency\_decay + leak)$$

11.3 Leakage and cross-talk

Let $\hat{x}_\ell = I_\ell(P_\ell(x))$ be reconstruction from layer ℓ.

$$r(x) = x - \sum_{\ell \in L} w'_\ell \hat{x}_\ell$$

$$Leak(x) = \frac{\|r(x)\|}{\|x\| + \varepsilon}$$

$$XT_{\ell m}(x) = \frac{|\langle P_\ell(x), P_m(x)\rangle|}{\|P_\ell(x)\|\|P_m(x)\| + \varepsilon}$$

Leakage law

No executable route may pass mirror consensus if $Leak(x) > \tau_{Leak}$ or $\max XT_{\ell m}(x) > \tau_{XT}$.

11.4 Dual-consensus gate

For candidate route z define:

$$Gate_{dual}(z) = Gate_{por}(z) \wedge Gate_{mirror}(z) \wedge Gate_{tempo}(z) \wedge Gate_{risk}(z)$$

$$\Gamma(z) = a_\psi \psi + a_\rho \rho - a_\omega(1 - \omega) - a_L Leak(z)$$

with hysteresis:

$$G(z) = \begin{cases} 1 & \text{if } g = 0 \wedge \Gamma(z) \geq \theta_{open} \\ 0 & \text{if } g = 1 \wedge \Gamma(z) \leq \theta_{close} \\ g & \text{otherwise} \end{cases}$$

11.5 Net edge

For a cycle $Z = (e_1, \ldots, e_m)$:

$$\Pi_Z = \prod_{i=1}^{m} p_i$$

$$NetEdge_Z = \Pi_Z - 1 - \sum_{i=1}^{m}(f_i + s_i(q_i))$$

Admissibility requires:

$$NetEdge_Z \geq \tau_{edge}$$

12 Temporal multiplexing core

12.1 Tri-carrier semantics

The scheduler is governed by:

1. t_1: extrinsic irreversible commit time,
2. t_2: intrinsic exploratory time,
3. ϕ: cyclic phase carrier.

12.2 Window lattice

Let phase windows be indexed by $j \in \{0, \dots, M-1\}$.

$$j(x) = \left\lfloor \frac{M\tilde{\phi}(x)}{2\pi} \right\rfloor$$

$$k(x) = (j(x), w(x) \bmod M)$$

12.3 Temporal key

Every consequential event **MUST** carry:

1. commit tick,
2. intrinsic tick,
3. phase bin,
4. wind modulus,
5. freshness class.

Temporal key law

No event affecting execution, hedge, calibration, promotion, replay, rollback, safe-hold, kill, or resource posture may omit a temporal key.

13 Nullcenter and jump legality

13.1 Nullcenter object

Let NC be the canonical nullcenter with maps:

$$\iota : \Sigma \to NC, \qquad \pi : NC \to \Sigma$$

An operator A factors through the nullcenter if:

$$A = \pi \circ \alpha \circ \iota$$

Nullcenter law

All jumps from exploration into commitment, and all accepted promotions or rollbacks, must factor through the nullcenter.

13.2 Jump legality

A jump is legal only if:

1. regime gate permits execution,
2. mirror gate passes,
3. temporal gate passes,
4. risk gate passes,
5. CSP quorum is valid where applicable,
6. nullcenter certificate exists,
7. witness event is appended before commit finalization.

14 Gate hierarchy

14.1 Canonical gate order

For a consequential execution path, gates are evaluated in this order:

1. **Regime Gate**
2. **Mirror Gate**
3. **Temporal Gate**
4. **Risk Gate**
5. **Quorum Gate**
6. **Actuation Gate**

For a calibration promotion path, gates are:

1. **Replay Gate**
2. **Benchmark Gate**
3. **Stability Gate**
4. **Promotion Gate**
5. **Live Activation Gate**

For a rollback path, gates are:

1. **Rollback Target Gate**
2. **Replay Consistency Gate**
3. **Integrity Restoration Gate**
4. **Reactivation Gate**

Gate precedence law

If any earlier gate in a path fails, later gates do not rescue the action. The earliest failing gate is the blocking cause and must be emitted explicitly.

15 Failure taxonomy

15.1 Canonical failure classes

The system recognizes the following failure classes:

ID	Name	Meaning	Severity
F-01	CandidateFailure	route candidate fails hard filters	local
F-02	QuorumFailure	CSP intent cannot assemble valid quorum	local
F-03	ExecutionFailure	lockstep execution fails or partially fails	high
F-04	HedgeFailure	hedge cannot arm, trigger, or unwind correctly	high
F-05	ReplayFailure	deterministic replay diverges or cannot proceed	blocking
F-06	IntegrityFailure	invariant or hash-chain violation	blocking
F-07	SchemaFailure	artifact schema mismatch or migration incompatibility	blocking
F-08	PromotionFailure	config promotion invalid or regressive	high
F-09	TemporalFailure	stale or invalid temporal key / TTL / freshness logic	blocking
F-10	ExternalDependencyFailure	provider dependency unavailable or inconsistent	degraded/high
F-11	ResourceFailure	CPU, memory, storage, or network budget exceeded	degraded/blocking

Failure law

Every failure must be classified, typed, severity-scored, and emitted as a structured failure artifact or event.

16 Recovery model

16.1 Recovery classes

Recovery paths are:

1. **Local recover**
2. **Safe-hold**
3. **Rollback**
4. **Kill**

16.2 Recovery routing

Failure class	Default recovery	Notes
CandidateFailure	local recover	reject candidate, continue loop
QuorumFailure	local recover	append witness, reset CSP
ExecutionFailure	safe-hold or rollback	depends on partial exposure and receipt state
HedgeFailure	safe-hold	may escalate to kill if exposure uncontrolled
ReplayFailure	safe-hold	no promotion or live action allowed
IntegrityFailure	safe-hold or kill	kill if hash-chain corruption irrecoverable
SchemaFailure	rollback or safe-hold	migration may be blocked
PromotionFailure	rollback	revert to prior promoted config
TemporalFailure	safe-hold	freeze actuation until temporal validity restored
ExternalDependencyFailure	degrade or safe-hold	live mode may pause while paper mode remains valid
ResourceFailure	degrade, safe-hold, or rollback	depends on whether legality can still be preserved

Recovery law

Recovery selection is not discretionary. It must follow typed failure class and explicit recovery routing unless a stricter policy overrides it.

17 Safe-hold and kill semantics

17.1 Safe-hold

Safe-hold means:

1. no new consequential execution,
2. no live promotion activation,
3. observation remains enabled,
4. replay remains enabled,
5. audit and diagnostic emission remain enabled,
6. rollback evaluation remains allowed.

17.2 Kill

Kill means:

1. all actuation disabled,
2. all promotion activity disabled,

3. no autonomous recovery without explicit restart path,
4. audit emission remains required until shutdown.

Safe-hold law

Safe-hold is the default blocking posture whenever blocking invariants fail but forensic continuity remains intact.

18 Rollback model

18.1 Rollback target

A rollback target is a prior promoted configuration or replay checkpoint that satisfies:

1. hash-chain continuity,
2. schema compatibility,
3. replay admissibility,
4. governance admissibility where required.

18.2 Rollback artifact

Every rollback must emit:

1. rollback ID,
2. source version,
3. target version,
4. failure trigger,
5. validation evidence,
6. post-rollback replay result.

Rollback law

No rollback may be treated as complete without a successful replay consistency check against the chosen target.

19 Artifact schema versioning

19.1 Version rule

Every artifact family has an explicit schema version. Schema version changes are:

1. **compatible**,
2. **migrated**,
3. **breaking**.

19.2 Migration artifact

A migration artifact **MUST** include:

1. source schema version,
2. target schema version,
3. transformation digest,
4. migration timestamp,
5. migration validity result.

Schema law

No schema migration may destroy replay lineage or hash-trace continuity. If preservation is impossible, the affected lineage must be quarantined and declared non-canonical.

20 Resource budgets

20.1 Canonical deployment classes

The system explicitly supports:

1. **Laptop Class**,
2. **Single VPS Class**,
3. **Single Node Class**.

20.2 Canonical resource envelopes

The core system **SHOULD** remain viable within these rough envelopes:

Resource	Target envelope	Interpretation
CPU	2–8 cores	no architectural need for cluster-scale parallelism
RAM	4–16 GB	bounded in-memory working set
Persistent storage	20–200 GB	rotating artifacts and compressed replay lineage
Network	moderate retail connectivity	no assumption of institutional bandwidth
Operator count	1	solo-operable by design

Infrastructure law

If a feature requires enterprise infrastructure to remain correct, then it is not part of the canonical core and must be moved to an optional profile or removed.

21 Resource posture and degradation

21.1 Resource hierarchy

When resources become constrained, the system degrades in this order:

1. reduce diagnostic verbosity,
2. reduce noncritical metric frequency,
3. reduce candidate breadth,
4. reduce trumpet layer breadth,
5. reduce calibration cadence,
6. preserve gates, replay-critical events, receipts, and integrity checks at all costs.

Degradation law

Resource degradation may reduce breadth, cadence, or convenience, but must never weaken:
1. blocking gates,
2. event completeness for consequential actions,
3. hash-chain integrity,
4. replay sufficiency,
5. rollback sufficiency.

22 Low-infrastructure persistence doctrine

22.1 Persistence model

The canonical persistence strategy is:

1. append-only local artifact streams,
2. rotation by time or size,
3. compressed archives for stale segments,
4. optional cold-copy backup,
5. no mandatory clustered storage layer.

Persistence law

No compaction or archival process may remove the minimum artifact set required for replay, promotion validation, rollback validation, and forensic reconstruction.

23 Dependency minimization doctrine

23.1 Canonical dependency classes

The repository may depend on:

1. Rust crates required for deterministic numerics, serialization, hashing, CLI, and async runtime,
2. venue adapters or HTTP clients where required,

3. optional compression libraries,
4. optional lightweight embedded storage behind feature flags.

23.2 Forbidden dependency inflation

The canonical core **MUST NOT** require:

1. container orchestration,
2. distributed brokers,
3. multi-node coordination services,
4. managed stream processors,
5. external telemetry platforms,
6. heavyweight ML infrastructure.

Dependency law

Any new mandatory dependency must prove that it increases correctness or integrity more than it increases infrastructure burden.

24 Repository realization contract

24.1 Canonical workspace structure

The coding agent **MUST** produce the following repository layout unless a deviation is explicitly justified in `README.md`:

Listing 2: Rust workspace v2.5.0

```
fixpoint-swarm-r/
  Cargo.toml
  README.md
  SPECIFICATION.md
  AGENTS.md
  .gitignore
  rust-toolchain.toml
  crates/
    fsr-types/
    fsr-fixed/
    fsr-time/
    fsr-nullcenter/
    fsr-trm/
    fsr-hypertorus/
    fsr-gate/
    fsr-mirror/
    fsr-candidates/
    fsr-csp/
    fsr-hedge/
    fsr-calibration/
    fsr-shadow/
    fsr-commit/
    fsr-integrity/
```

```
    fsr-resource/
    fsr-execution/
    fsr-sim/
    fsr-runtime/
    fsr-cli/
  config/
    default.yaml
    conservative.yaml
    balanced.yaml
    aggressive.yaml
    laptop.yaml
    vps.yaml
    single-node.yaml
  tests/
    integration/
  fixtures/
    synthetic/
    replay/
    orderbooks/
    promotions/
  ci/
    validate-config.sh
    run-paper-suite.sh
    run-replay-suite.sh
    run-promotion-suite.sh
    run-integrity-suite.sh
    run-resource-suite.sh
  docs/
    runbooks/
```

24.2 Mandatory top-level files

The coding agent **MUST** produce:

1. `Cargo.toml`
2. `README.md`
3. `SPECIFICATION.md`
4. `AGENTS.md`
5. `rust-toolchain.toml`
6. at least one CI entrypoint script under `ci/`

25 Crate realization contract

25.1 Crate responsibilities

Crate	Responsibility
fsr-types	Shared types, enums, IDs, requests, receipts, artifact models

fsr-fixed	Deterministic numerics, basis points, ratios, fixed-point helpers
fsr-time	Tri-carrier clocks, temporal keys, validity windows, commitments
fsr-nullcenter	Factor-through typing, jump certificates, windnarbe, legality checks
fsr-trm	Resonance metrics, coherence, entropy, sync, phase impulse
fsr-hypertorus	Window scheduler, phase bins, lattice addressing, phase damping
fsr-gate	Regime logic, Spiral Index, temporal and risk admissibility
fsr-mirror	MCI, seam checks, dual-consensus gate, mirror route verification
fsr-candidates	Route discovery, filters, ranking, compressed family selection
fsr-csp	Intent, quorum, lockstep, abort, reset, execution legality
fsr-hedge	Hedge FSM, sizing, trigger rules, recovery logic
fsr-calibration	Benchmarking, contraction, update proposals, acceptance and promotion checks
fsr-shadow	Shadow-Chain operational stream
fsr-commit	Commitment Chain, crystallized receipts, promoted configs
fsr-integrity	Invariant checks, failure classification, rollback and safehold logic, schema posture
fsr-resource	Resource budgets, posture classification, degradation control, compaction policy
fsr-execution	Paper broker, venue traits, live stubs, route requests
fsr-sim	Synthetic regimes, slippage, TTL, partial fills, latency injection
fsr-runtime	Orchestration loop, health, metrics, task wiring
fsr-cli	Commands for paper run, replay, audit inspection, config validation

25.2 Crate boundary law

Crate law

Shared domain types must live in `fsr-types`. Decision-relevant numerics must live in `fsr-fixed`. No cyclic crate dependency is allowed. Traits define boundaries; hidden cross-crate state is forbidden.

26 Mandatory Rust type contract

Listing 3: Core type sketch

```
pub enum RegimeState { Alpha, Beta, Gamma }

pub enum HedgeState { Safe, Armed, Triggered, Disarmed }

pub enum PorState { Search, Lock, Verify, Commit }
```

```
pub enum CspState {
    Idle,
    Discovered,
    IntentOpen,
    IntentQuorum,
    ExecLockstep,
    Confirm,
    Settled,
    Abort,
    Reset,
}

pub enum PromotionState {
    Candidate,
    PaperValidated,
    PromotionPending,
    LiveEligible,
    LiveActive,
    Demoted,
    Revoked,
}

pub enum IntegrityPosture {
    Healthy,
    Degraded,
    SafeHold,
    RollbackPending,
    Killed,
}

pub enum ResourcePosture {
    Nominal,
    Constrained,
    Scarce,
    Emergency,
}

pub struct TemporalKey {
    pub commit_tick: u64,
    pub intrinsic_tick: u64,
    pub phase_bin: u16,
    pub wind_mod: u16,
    pub freshness_class: FreshnessClass,
}

pub struct ResonanceSnapshot {
    pub kappa: Q32,
    pub entropy: Q32,
    pub sync: Q32,
    pub m: Q32,
    pub si: Q32,
    pub leak: Q32,
    pub mci: Q32,
}
```

```
pub struct NullcenterCert {
    pub gate_open: bool,
    pub phi_delta: Q32,
    pub wind_count: u64,
    pub pre_digest: Hash256,
    pub post_digest: Hash256,
}
```

Type law

The coding agent may extend these types, but must not weaken or erase the canonical state enum structure.

27 Mandatory trait contract

Listing 4: Trait sketch

```
pub trait NullcenterFactored: Send + Sync {
    fn factor(&self, state: &State) -> Result<State, NcError>;
}

pub trait MirrorConsensus: Send + Sync {
    fn mci(&self, candidate: &CandidateCycle) -> Result<Q32, MirrorError>;
    fn allow(&self, candidate: &CandidateCycle) -> Result<bool, MirrorError>;
}

pub trait KairosScheduler: Send + Sync {
    fn schedule(&self, state: &State) -> ScheduleOutput;
}

pub trait CalibrationShell: Send + Sync {
    fn benchmark(&self, cfg: &StrategyConfig) -> BenchmarkReport;
    fn propose_update(&self, cfg: &StrategyConfig) -> StrategyConfig;
    fn accept_update(&self, old: &StrategyConfig, new: &StrategyConfig) -> bool;
}

pub trait ShadowSink: Send + Sync {
    fn append(&self, evt: ShadowEvent) -> Result<(), ShadowError>;
}

pub trait CommitSink: Send + Sync {
    fn append_commit(&self, evt: CommitEvent) -> Result<(), CommitError>;
}

pub trait PromotionGate: Send + Sync {
    fn evaluate(&self, proposal: &PromotionProposal) -> PromotionDecision;
}

pub trait IntegrityController: Send + Sync {
    fn classify_failure(&self, failure: FailureArtifact) -> RecoveryAction;
    fn enter_safe_hold(&self) -> Result<(), IntegrityError>;
    fn rollback(&self, target: RollbackTarget) -> Result<RollbackResult,
   IntegrityError>;
```

```
}

pub trait ResourceGovernor: Send + Sync {
    fn posture(&self, snap: &ResourceSnapshot) -> ResourcePosture;
    fn degrade(&self, posture: ResourcePosture) -> DegradationPlan;
}
```

Trait law

The coding agent may refine trait signatures, but the repository must preserve these conceptual interfaces and boundaries.

28 Artifact contracts

28.1 Mandatory artifact families

The repository **MUST** produce machine-readable artifacts in these families:

1. events,
2. snapshots,
3. candidate traces,
4. quorum reports,
5. execution receipts,
6. hedge reports,
7. calibration reports,
8. promotion proposals,
9. promotion decisions,
10. rollback reports,
11. replay reports,
12. audit summaries,
13. schema migration reports,
14. integrity status reports,
15. resource status reports.

28.2 Mandatory minimum fields

Every artifact **MUST** include:

1. artifact type,
2. artifact ID,
3. creation timestamp or tick reference,
4. run ID,
5. schema version,

6. digest or hash,
7. origin module.

Artifact law

If an artifact influences replay, execution legality, promotion, rollback, or integrity posture, then its schema must be explicit and versioned in code.

29 Snapshot contracts

29.1 Mandatory snapshot classes

1. `MarketSnapshot`
2. `ResonanceSnapshot`
3. `CandidateSnapshot`
4. `RegimeSnapshot`
5. `InventorySnapshot`
6. `CalibrationSnapshot`
7. `PromotionSnapshot`
8. `IntegritySnapshot`
9. `ResourceSnapshot`
10. `ReplayCheckpoint`

Snapshot law

Every snapshot used for decision, replay, validation, promotion, rollback, integrity evaluation, or resource posture evaluation must be serializable, hashable, and independent of hidden process-local state.

30 Candidate evaluation law

The hard filter order is:

1. structural validity,
2. depth sufficiency,
3. slip cap,
4. fee-adjusted edge,
5. latency feasibility,
6. inventory admissibility,
7. risk cap,
8. regime admissibility,
9. mirror admissibility,

10. temporal admissibility.

Candidate law

Ranking occurs only after all hard filters pass. No soft ranking may rescue a structurally, temporally, or mirror-inadmissible route.

31 Determinism and arithmetic

The core decision path **MUST** be deterministic. Use:

1. fixed-point for decision-relevant continuous quantities,
2. integer basis points for fees and slip caps,
3. integer microseconds for time,
4. integer lot or step sizes for quantities.

Arithmetic law

Floating-point is forbidden in the decision-critical path unless reduced to a deterministic contract before affecting gates, execution, hedge, promotion, rollback, integrity, or resource-governed decisions.

32 Configuration schema

32.1 Profiles

The repository ships:

1. `default`,
2. `conservative`,
3. `balanced`,
4. `aggressive`,
5. `laptop`,
6. `vps`,
7. `single-node`.

32.2 Canonical YAML

Listing 5: Canonical configuration YAML

```yaml
supervisor:
  leverage_cap: 5
  risk_budget: 0.10
  regime:
    kappa_min: 0.60
    entropy_max: 1.20
    open_hysteresis: 0.02
    close_hysteresis: 0.01
```

```
temporal:
  phase_bins: 24
  near_window_ms: 150
  strategic_window_ms: 1000

mirror:
  mci_min: 0.80
  leak_max: 0.08
  xt_max: 0.20

arb:
  enable: true
  max_cycle_len: 4
  tau_edge: 0.0005
  ttl_ms: 150
  max_slip_bps: 0.5
  min_size: 1000
  cex_order_type: FOK
  cex_ioc_fallback: false
  dex_atomic_only: true

hedge:
  enable: true
  alpha: 0.70
  dd_max: 0.03
  rec_dd: 0.01
  tp_bps: 15
  sl_bps: 20

calibration:
  enable: true
  max_step_per_update: 0.05
  min_stability: 0.75
  min_efficiency: 0.60
  live_promotion_requires_human_approval: true

integrity:
  safe_hold_on_replay_divergence: true
  kill_on_hash_chain_corruption: true
  rollback_on_promotion_regression: true

resource:
  max_candidate_breadth: 256
  max_active_layers: 4
  max_snapshot_rate_hz: 10
  compact_noncritical_metrics: true
  emergency_freeze_calibration: true

runtime:
  loop_ms: 100
  shadow_rotate_daily: true
  replay_strict: true
```

33 Implementation phases

33.1 Mandatory implementation order

The coding agent **MUST** implement in this order:

1. fsr-types
2. fsr-fixed
3. fsr-shadow
4. fsr-commit
5. fsr-time
6. fsr-nullcenter
7. fsr-trm
8. fsr-hypertorus
9. fsr-gate
10. fsr-mirror
11. fsr-candidates
12. fsr-csp
13. fsr-hedge
14. fsr-calibration
15. fsr-integrity
16. fsr-resource
17. fsr-execution
18. fsr-sim
19. fsr-runtime
20. fsr-cli

33.2 Phase closure rule

A phase is complete only if:

1. the crate compiles,
2. crate-level tests pass,
3. public interfaces are documented,
4. dependencies remain non-cyclic,
5. no required canonical type or trait has been violated.

34 Testing constitution

34.1 Mandatory test families

The repository **MUST** include:

1. unit tests for arithmetic and fixed-point behavior,
2. unit tests for state machine transitions,
3. unit tests for gate logic,
4. integration tests for CSP,
5. integration tests for Mirror-Hedge,
6. replay determinism tests,
7. promotion validation tests,
8. rollback validation tests,
9. integrity and resource degradation tests,
10. end-to-end paper mode tests.

34.2 Regression rule

Regression law

Every bug fix affecting execution, replay, promotion, rollback, or integrity must add at least one regression test.

35 CLI contract

The repository **MUST** ship at least:

1. `fsr-cli run-paper -config config/conservative.yaml`
2. `fsr-cli replay -shadow <path>`
3. `fsr-cli audit-inspect -shadow <path>`
4. `fsr-cli validate-config -config <path>`
5. `fsr-cli validate-promotion -proposal <path>`
6. `fsr-cli rollback-check -target <path>`
7. `fsr-cli resource-posture -config <path>`

CLI law

The CLI is part of the canonical system surface. A repository without these commands or clearly equivalent commands is incomplete.

36 Documentation contract

The coding agent **MUST** produce:

1. `README.md` explaining build, test, paper mode, replay, audit inspection, promotion validation, rollback validation
2. module-level rustdoc for every crate
3. runbooks under `docs/runbooks/`
4. a `SPECIFICATION.md` copy or derivative of this specification suitable for repository use

Documentation law

The repository is incomplete if another agent cannot build and operate it from repository-local documentation alone.

37 Runbook requirements

The repository **MUST** include operator-facing runbooks for:

1. quorum failure burst,
2. replay divergence,
3. promotion regression,
4. integrity safe-hold entry,
5. rollback activation,
6. kill activation and recovery prerequisites,
7. resource emergency degradation.

38 CI contract

The CI surface **MUST** include:

1. workspace build,
2. unit tests,
3. integration tests,
4. replay determinism suite,
5. paper simulation suite,
6. promotion validation suite,
7. integrity suite,
8. resource suite,
9. config validation.

CI law

A repository cannot claim spec-conformant readiness if replay, paper validation, promotion validation, integrity validation, and resource validation are not all present in CI.

39 Acceptance tests

The repository must prove:

1. identical replay under fixed seeds and identical artifacts,
2. no execution without a nullcenter certificate,
3. no hanging CSP states,
4. mirror-inconsistent candidates are rejected,
5. calibration shell only accepts evidence-bearing updates,
6. promotion candidates cannot skip paper validation,
7. rollback targets are replay-valid before rollback completion,
8. net PnL after frictions is non-negative in baseline paper suites,
9. hedge lowers drawdown in adverse synthetic regimes,
10. scheduler remains deterministic across runs,
11. audit output is inspectable and sufficient for reconstruction,
12. safe-hold freezes actuation without destroying observation and audit continuity,
13. resource scarcity degrades breadth but not legality,
14. repository-local documentation is sufficient for autonomous use by another agent.

40 Definition of done

The repository is done only if:

1. `cargo build -workspace` passes,
2. `cargo test -workspace` passes,
3. paper mode runs end-to-end,
4. replay reproduces decisions,
5. all jump, commit, hedge, calibration, promotion, rollback, integrity, and resource events are audit-inspectable,
6. CLI usage is sufficient for autonomous use by another agent,
7. CI proves conformance for replay, paper, promotion, integrity, and resource paths,
8. the canonical core does not require enterprise infrastructure to remain correct,
9. another coding agent could continue implementation or maintenance using only the repository contents.

41 Formal implementation checklist

Check	Question	Pass?
FSR25-01	Is the repository still pure Rust?	[ ]
FSR25-02	Are t_1, t_2, and ϕ explicit wherever required?	[ ]
FSR25-03	Do all jumps, promotions, and rollbacks factor through nullcenter legality?	[ ]
FSR25-04	Is trumpet multiplexing finite and deterministic?	[ ]
FSR25-05	Does the dual-consensus gate enforce MCI and leakage bounds?	[ ]
FSR25-06	Is CSP deterministic, abort-safe, and transition-complete?	[ ]
FSR25-07	Are Shadow-Chain and Commitment Chain both operational and hash-consistent?	[ ]
FSR25-08	Is calibration evidence-bearing, promotion-gated, rollback-capable, and resource-aware?	[ ]
FSR25-09	Does replay reconstruct temporal keys, windnarbe, promotion lineage, rollback lineage, and resource posture?	[ ]
FSR25-10	Can an autonomous coding agent implement, run, validate, and maintain the repository from this specification and repository-local documentation alone?	[ ]

42 Closing doctrine

Final doctrine

The decisive progression in this version is that the specification becomes directly executable as an implementation mandate for a coding agent. FIXPOINT SWARM-R v2.5.0 preserves the nullcenter-governed temporal multiplexing core, the operating legality of v2.2.0, the integrity and recovery order of v2.3.0, and the leverage-under-scarcity doctrine of v2.4.0, but now binds the entire system to a repository-realization contract. The result is the same architecture on a higher maturity level: not merely a strong design, but a document that can be handed directly to an autonomous coding agent so that the software can be realized without hidden assumptions, hidden architecture, or hidden operator knowledge.

End of specification FIXPOINT SWARM-R v2.5.0